

LAB IMPLEMENTATION 7

Smart Contract Deployment Using Truffle and Ganache

Aim

To establish a local blockchain environment using Ganache and connect it with MetaMask and Remix IDE. To compile, deploy, and interact with a Solidity smart contract using a Ganache account.

Theory

A **smart contract** is a program deployed on a blockchain that executes functions according to its defined rules. In this experiment, Ganache provides a local Ethereum-compatible blockchain with development accounts, while MetaMask manages an account and connects to the local network. Remix IDE is used to write, compile, deploy, and interact with the Solidity contract.

The contract stores a number and maintains an array of records containing an identifier, title, sender address, and timestamp. Functions marked `view` read blockchain state without creating a transaction, while state-changing functions create transactions that consume gas and are recorded in blocks. Contract events provide log entries associated with state-changing operations. The deployment and transaction information can be inspected in the Ganache environment.

Steps

1. Install and start Ganache to create a local Ethereum-compatible blockchain.
2. Add the Ganache network in MetaMask using the local RPC endpoint and chain ID shown in the environment.
3. Import a Ganache account into MetaMask using its development private key.
4. Open Remix IDE and load `Lab7Storage.sol` containing the Solidity smart contract.
5. Select the Solidity compiler, compile the contract, and verify that `Lab7Storage` is compiled successfully.
6. Select the Ganache provider in Remix, choose the compiled `Lab7Storage` contract, and deploy it using the connected account.
7. Inspect the deployed contract functions in Remix and verify the deployment blocks and transaction details in Ganache.
8. Interact with the contract through its available functions and observe the corresponding blockchain state and transaction information.

Code

```
1 // SPDX-License-Identifier: GPL-3.0
2
```

```
3  /*
4  * =====
5  * Experiment No : 7
6  * AIM          : Introduction to Truffle, Establish local
7  *                blockchain using Truffle/Ganache and
8  *                deployment of Smart Contract
9  *
10 * Author   : Jay Kerkar
11 * Roll No  : 16
12 * Batch    : A
13 *
14 * NOTE: This is a simple record-keeping contract. Rename the
15 * contract/fields to match your chosen Mini Project.
16 * =====
17 */
18
19 pragma solidity >=0.7.0 <0.9.0;
20
21 /// @title Simple record storage (Author: Jay Kerkar | Roll No: 16 | Batch: A)
22 contract Lab7Storage {
23
24     // Author details stored on-chain as proof of ownership
25     string public constant AUTHOR = "Jay Kerkar";
26     string public constant ROLL_NO = "16";
27     string public constant BATCH = "A";
28
29     // Account that deployed the contract (the Ganache account selected in MetaMask
30     )
31     address public owner;
32
33     // A single number that can be stored and retrieved (like Storage.sol)
34     uint256 private number;
35
36     // A record entry
37     struct Record {
38         uint256 id;
39         string title;
40         address addedBy;
41         uint256 timestamp;
42     }
43
44     Record[] private records;
45
46     // Events show up in the transaction logs inside Ganache
47     event NumberStored(address indexed by, uint256 value);
48     event RecordAdded(uint256 indexed id, string title, address indexed addedBy);
49
50     constructor() {
51         owner = msg.sender;
52     }
53
54     /// @notice Store a number (this is a transaction -> costs gas, creates a block
55     in Ganache)
56     function store(uint256 num) public {
57         number = num;
58         emit NumberStored(msg.sender, num);
59     }
60 }
```

```
59  /// @notice Read the stored number (view call -> free, no block mined)
60  function retrieve() public view returns (uint256) {
61      return number;
62  }
63
64  /// @notice Add a new record (transaction)
65  function addRecord(string memory title) public {
66      uint256 id = records.length;
67      records.push(Record(id, title, msg.sender, block.timestamp));
68      emit RecordAdded(id, title, msg.sender);
69  }
70
71  /// @notice Get a record by id (view call)
72  function getRecord(uint256 id)
73      public
74      view
75      returns (uint256, string memory, address, uint256)
76  {
77      require(id < records.length, "Record does not exist");
78      Record memory r = records[id];
79      return (r.id, r.title, r.addedBy, r.timestamp);
80  }
81
82  /// @notice Total number of records (view call)
83  function totalRecords() public view returns (uint256) {
84      return records.length;
85  }
86  }
```

Listing 1. Lab7Storage.sol

Output

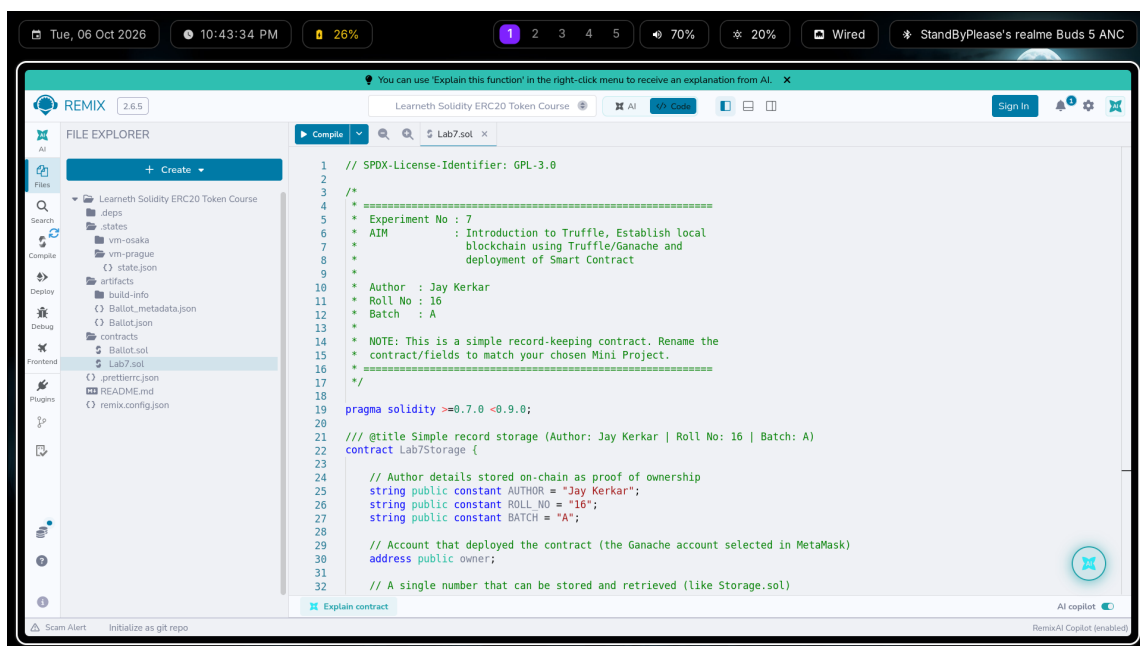


Fig. 1. Solidity contract loaded in Remix IDE

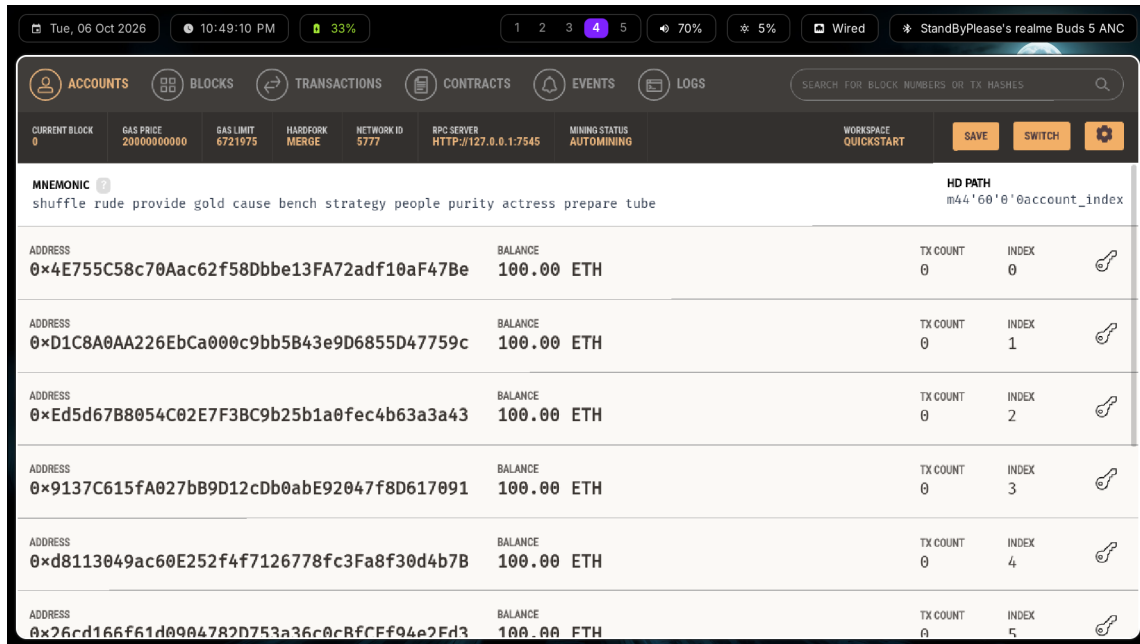


Fig. 2. Ganache development accounts with initial balances

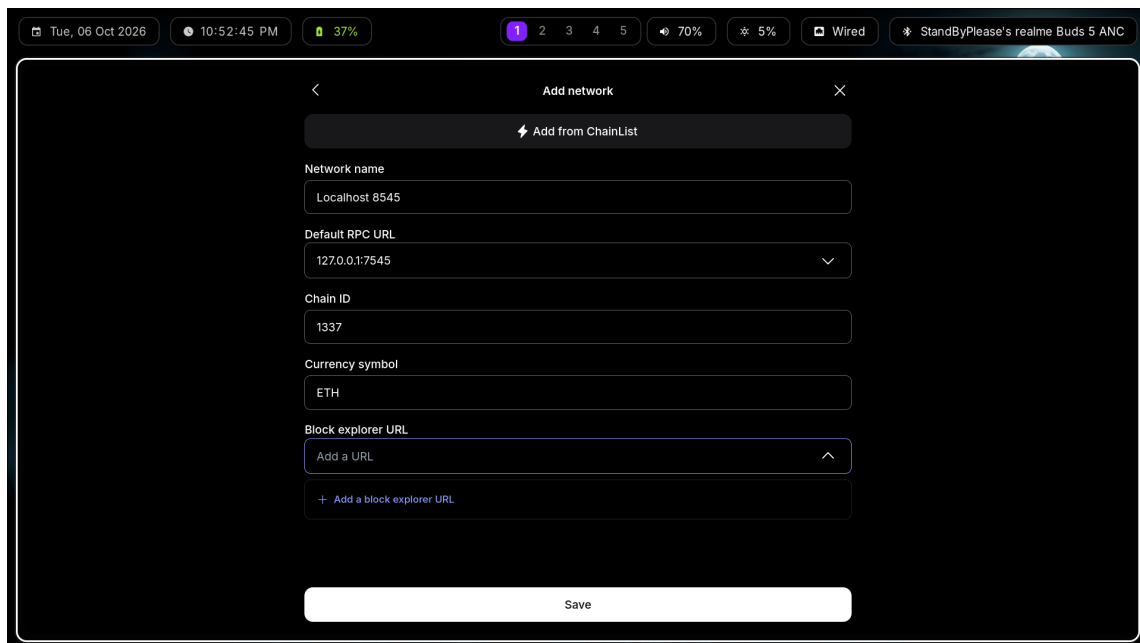


Fig. 3. MetaMask network configured for Localhost 8545



Fig. 4. Ganache account address and development private key

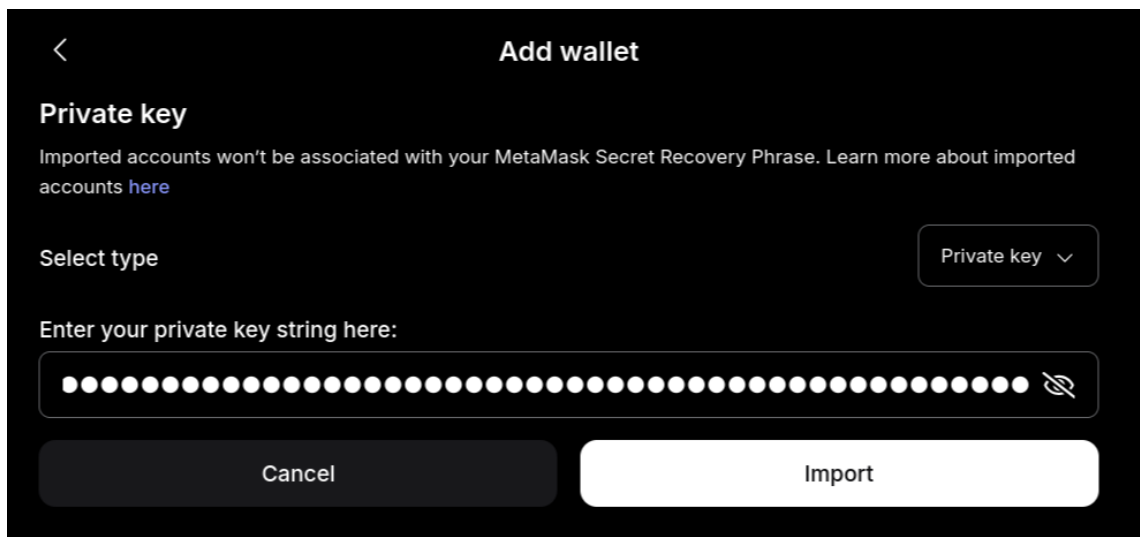


Fig. 5. MetaMask private key import screen

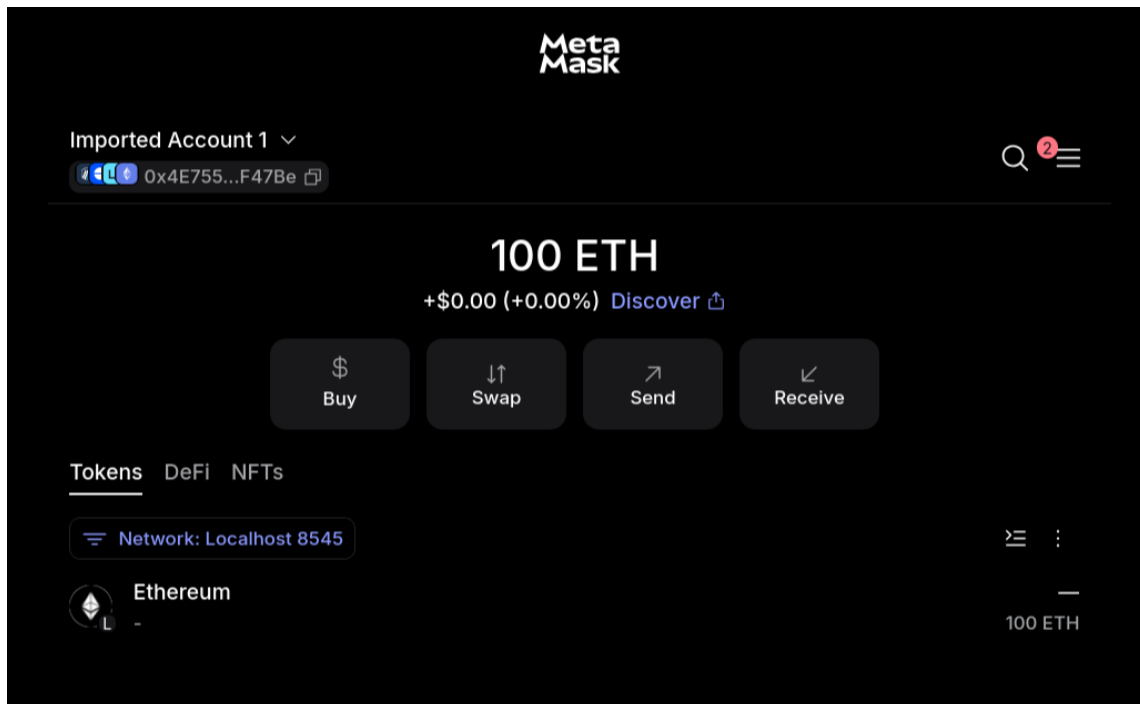


Fig. 6. Imported MetaMask account showing 100 ETH

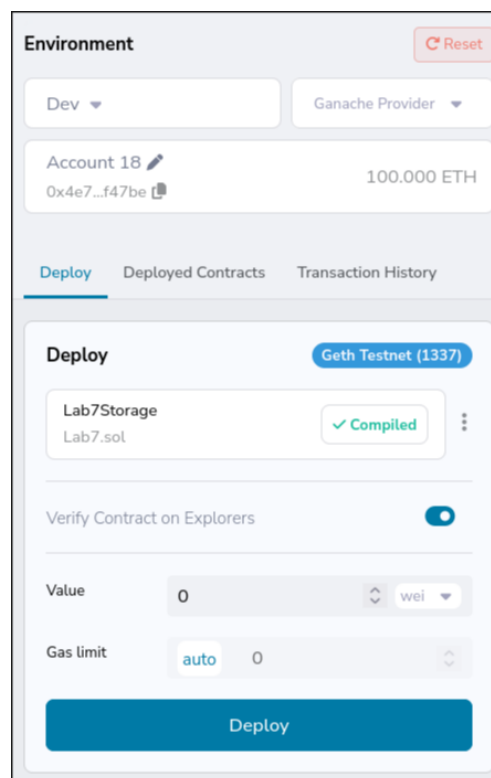


Fig. 7. Remix deployment panel with Ganache provider

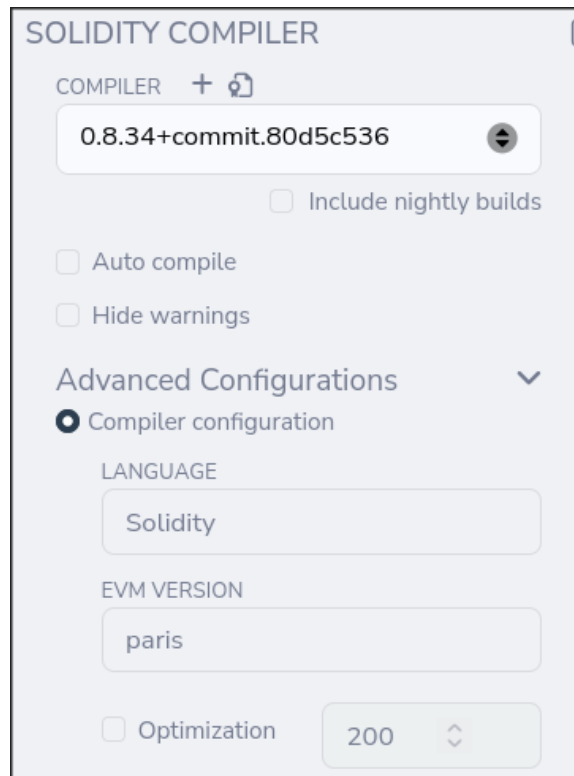


Fig. 8. Solidity compiler configured for contract compilation

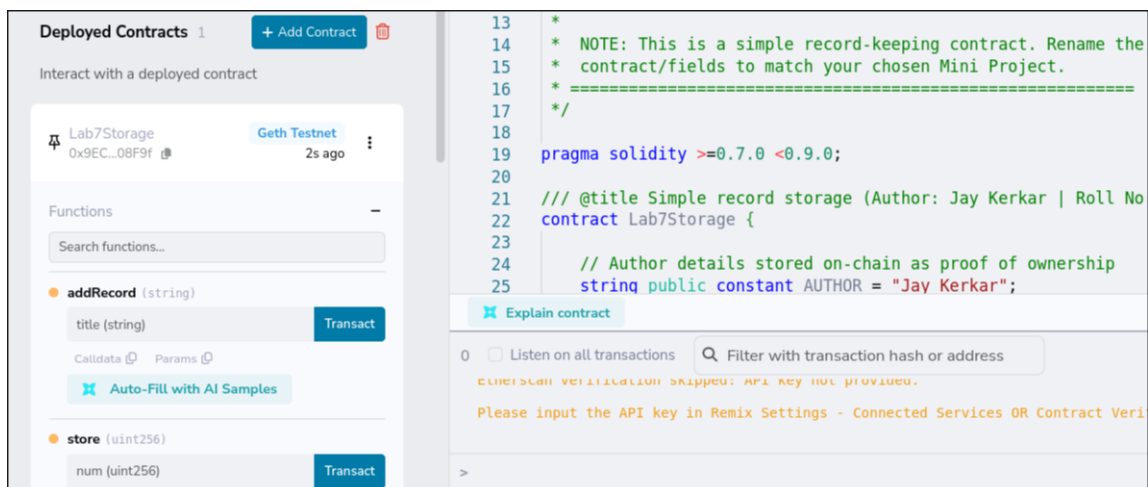


Fig. 9. Deployed contract functions visible in Remix

ACCOUNTS	BLOCKS	TRANSACTIONS	CONTRACTS	EVENTS	LOGS	SEARCH FOR BLOCK NUMBERS OR TX HASHES
CURRENT BLOCK 2	GAS PRICE 2000000000	GAS LIMIT 6721975	HARDFORK MERGE	NETWORK ID 5777	RPC SERVER HTTP://127.0.0.1:7545	MINING STATUS AUTOMINING
WORKSPACE QUICKSTART	SAVE	SWITCH	SETTINGS			
BLOCK 2	MINED ON 2026-10-06 23:02:41	GAS USED 792588				1 TRANSACTION
BLOCK 1	MINED ON 2026-10-06 23:00:12	GAS USED 3000000				1 TRANSACTION
BLOCK 0	MINED ON 2026-10-06 22:48:59	GAS USED 0				NO TRANSACTIONS

Fig. 10. Ganache blocks showing contract transactions

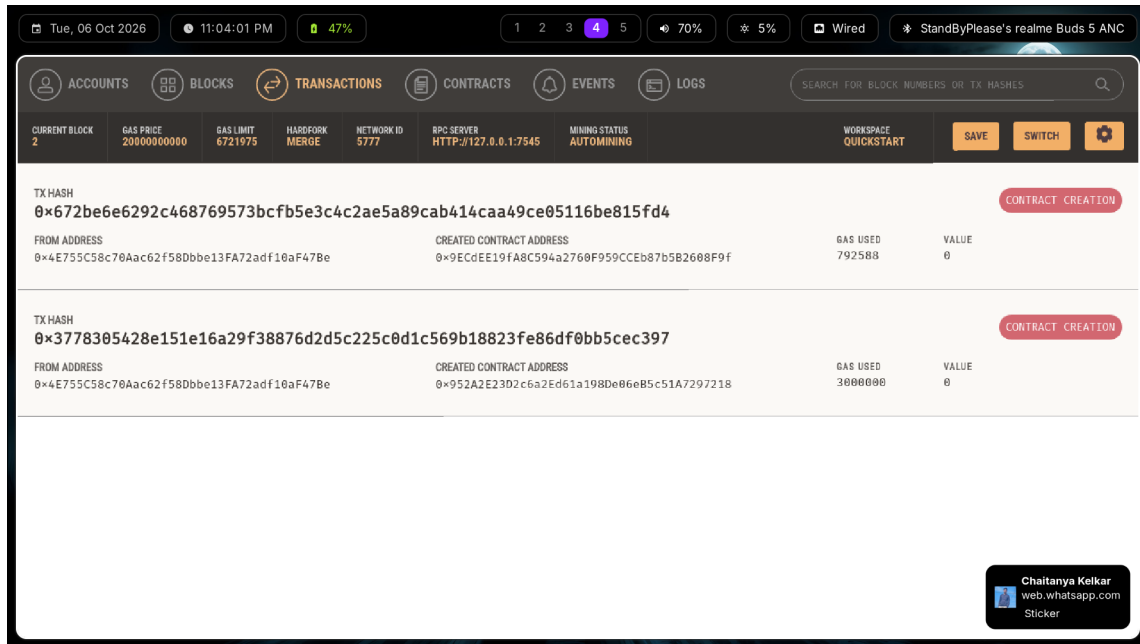


Fig. 11. Ganache transaction list showing contract creation

Conclusion

The local blockchain environment was established using Ganache and connected to MetaMask and Remix IDE. The Lab7Storage smart contract was compiled and deployed through the Ganache provider, and its functions were displayed in Remix. The screenshots showed the configured local network, imported account, deployed contract, mined blocks, and contract creation transactions. This demonstrated the basic development workflow for deploying and interacting with a Solidity smart contract on a local blockchain.